MGMTMFE 431 — Homework 1

AI Prompts and Material Corrections

Authors
QIDIAN XING · MICHAEL SUHERMAN
GUS GUENTHER · ENDI ZONANA

1. How We Used AI

We used AI as a research and coding assistant throughout Homework 1. Rather than asking AI to produce the entire assignment in one step, we divided the project into the same modeling and coding stages required by the homework. For each stage, we specified the economic setting, notation, assumptions, data definitions, required outputs, and numerical checks.

AI was used to help derive the state-space representation, write and organize Python code, implement the Kalman filter and maximum-likelihood estimation, construct the augmented annual-observation model, produce forecasts, and interpret the results. We then checked the generated output using simulation with known parameters, numerical likelihood checks, consistency with the lecture equations, and repeated execution of the code.

The prompts below are the key working prompts used to guide the modeling and coding tasks. They are consolidated from the actual working instructions and preserve the substance of the prompts used during the assignment. A complete chat transcript is not included.

2. Key Prompts Used for the Homework

Prompt 0 — Monthly Model Specification

> **Prompt:**
>
> Act as a PhD-level econometrician and use the notation from Lecture 1. We want to model U.S. monthly inflation with a linear Gaussian state-space model.
>
> Let monthly inflation equal a constant intercept, a mean-zero latent AR(1) state, and an observation-noise term. Assume the state and observation shocks are independent Gaussian shocks with constant variances, all coefficients are time-invariant, and the latent process is stationary.
>
> Write the model using the notation: α_t for the latent state, $a_{t|s} = E(\alpha_t \mid y_{1:s}; \theta)$ for the conditional state mean, $P_{t|s}$ for the conditional state variance, μ for the constant intercept, η_t for the observation shock with variance H, ε_{t+1} for the state shock with variance Q.
>
> Give the observation equation, state equation, assumptions, and stationary initial distribution. Check that the notation and equations are consistent with Lecture 1 before moving to code.

Purpose: Establish the exact model, assumptions, notation, and initial distribution before implementation.

Prompt 1 — Coding Task 1: Simulate the Monthly Model

> **Prompt:**
>
> Using the monthly AR(1) state-space model already specified, write Python code to simulate data from the model.
>
> Use a parameter T_{sample} to control the sample length and place all parameter values and the random seed at the beginning so they are easy to change.
>
> Use: $\mu = 0.20$; $\varphi = 0.90$; $\sqrt{Q} = 0.20$; $\sqrt{H} = 0.10$
>
> Therefore: Q = 0.04; H = 0.01
>
> Sample the initial latent state from the stationary prior rather than setting the realized initial state equal to zero.

Return both the observed inflation series and the true latent state. Plot the simulated inflation series and the true latent component so we can visually check whether the simulation behaves as expected.

Purpose: Generate data for which the true parameters and true latent state are known, allowing the implementation to be checked before using real data.

Prompt 2 — Coding Task 2: Estimate Simulated Monthly Data

Prompt:

Extend the simulation code to estimate the model by maximum likelihood using the Kalman innovation likelihood and the stationary prior.

For every candidate parameter vector, recompute the stationary initialization and run the Kalman filter through the full sample. Estimate μ, φ, Q, and H subject to the required parameter restrictions.

Report: true parameter values, parameter estimates, standard errors, optimization convergence information.

Run the experiment for:
$T_{\text{sample}} = 300$; $T_{\text{sample}} = 3{,}000$; $T_{\text{sample}} = 300{,}000$
Keep the true parameters fixed and record the random seeds.

Compute parameter standard errors from the curvature of the actual log likelihood at the estimate. If optimization is carried out in transformed parameters, correctly transform the covariance matrix back to the reported parameter scale using the appropriate Jacobian or delta method.

Do not simply use an optimizer's internal approximate inverse Hessian without checking it.

Compare the estimates and standard errors across the three sample sizes. Under the correctly specified model, estimates should generally concentrate around the truth and standard errors should tend to shrink as the sample grows, although an individual estimate does not have to move monotonically closer to the truth.

Purpose: Verify that the estimator recovers known parameters and that the reported uncertainty behaves sensibly as sample size increases.

Prompt 3 — Coding Task 3: Estimate Actual Monthly Inflation

Prompt:

Replace the simulated observations with the supplied US_inflation47_25.csv data.

Use the complete historical sample from January 1947 through December 2024. Convert the supplied Inflation values to monthly percentage points by multiplying them by 100. Do not annualize the monthly observations.

Use the estimator that passed the simulation checks to estimate the monthly state-space model on the real data.

Report the estimated parameters and standard errors.

Create two figures:
1. observed monthly inflation together with the estimated intercept plus filtered latent state, $\hat{\mu} + a_{t|t}$;
2. filtered latent state $a_{t|t}$ and smoothed latent state $a_{t|T}$ only.
Report $\hat{\varphi}^{12}$ and explain what it says about the persistence of the latent monthly state over a one-year horizon.

Explain the difference between filtering and smoothing in terms of their information sets. Also discuss whether the results suggest possible model misspecification.

Purpose: Apply the validated monthly model to real inflation data and interpret the estimated latent component and persistence.

Prompt 4 — Coding Task 4: Construct the Augmented Annual-Observation Model

Prompt:

Define the rolling 12-month inflation sum as:
$z_t = y_t + y_{t-1} + \ldots + y_{t-11}$.

Keep the underlying monthly model unchanged, but now suppose that only z_t is observed.

Construct a 24-dimensional augmented state containing: the current and previous 11 monthly latent states, the current and previous 11 monthly observation shocks.

Write the complete augmented state-space model and clearly specify: the augmented observation equation, Z_{aug}, d_{aug}, T_{aug}, c_{aug}, R_{aug}, Q_{aug}, H_{aug}, the joint stationary initial mean, the full joint stationary initial covariance matrix.

Explain why the monthly observation shocks must be carried in the state and how this accounts for the overlap between adjacent rolling 12-month observations.

Explain why $H_{aug} = 0$ does not mean the original monthly observation-noise variance H is zero.

Check all matrix dimensions and verify that the transition matrix correctly shifts the 12 latent-state lags and the 12 observation-shock lags.

Purpose: Represent overlapping annual observations correctly while preserving the same underlying monthly data-generating process.

Prompt 5 — Coding Task 5: Estimate the Annual Model and Compare Forecasts

Prompt:

Implement the augmented 24-dimensional model and its Kalman innovation likelihood.

First validate the annual-observation estimator on simulated data: simulate 3,000 monthly observations from the known monthly model, construct rolling 12-month sums, estimate the augmented annual-observation model, compare the estimated parameters with the known truth.

Only after the simulation check passes, estimate the augmented model on the historical rolling 12-month inflation series.

Compare the empirical annual-observation estimates with the monthly-observation fit, including $\hat{\varphi}^{12}$, and explain the economic implications.

Next, compare forecasts from the two fitted models using exactly the same target:

$z_{t+12} = y_{t+1} + y_{t+2} + ... + y_{t+12}$.

For the monthly fit: use monthly observations through time t, use the filtered monthly state at the forecast origin.

For the annual-observation fit: use annual observations through time t, use the filtered augmented state at the forecast origin.

For both fits: propagate the model forward 12 months, do not update the state using intervening future observations, do not use smoothed states.

Use the same 913 target observations, from December 1948 through December 2024.

Also include the benchmark forecast:

$\hat{z}_{t+12|t} = z_t$.

Report RMSE and MAE for all three forecasting methods, align plots by target date, and explain which method performs better under each loss function.

Explicitly explain why the comparison is retrospective rather than a genuine out-of-sample test when the model parameters are estimated using the full sample.

Purpose: Validate the augmented estimator and compare the monthly and annual observation schemes fairly on the same entirely future 12-month inflation target.

Prompt 6 — Coding Task 6: Simulate from the Fitted Model and Diagnose Persistence

Prompt:

Use the monthly parameters estimated from the annual-observation fit to simulate four independent monthly samples, each with the same length as the historical data: 936 months.

For each simulated sample: start from the stationary prior, generate the monthly latent state and monthly inflation, construct the rolling 12-month inflation sums.

Plot each simulated annual path against the historical annual series using common axes so that differences in magnitude and persistence are visually comparable.

Do not expect the simulations to reproduce historical events in the same calendar years. Instead, compare their overall properties, especially the size and duration of inflation episodes.

Then calculate the historical sample correlation:

Corr(z_t, z_{t-12}).

Compare it with the corresponding correlation implied by: the monthly-observation fit, the annual-observation fit.

Clearly distinguish this observed annual lag-12 correlation from the latent-state persistence measure φ^{12}.

Discuss whether the annual-observation fit moves the model closer to the persistence observed in the data and whether important economic features still appear to be missing.

Purpose: Check whether a model that is numerically estimated correctly can also generate economically realistic persistence and inflation episodes.

3. Material Corrections to AI Output

Correction 1 — Kalman Innovation Timing

Original issue:

An AI-generated explanation stated that the innovation was calculated "before seeing the current observation." That wording was incorrect. The predictive distribution is formed before observing y_t, but the realized innovation itself requires the current observation.

Correction prompt:

Recheck the timing of the Kalman innovation against the lecture equations.

The predictive distribution should be formed using information through t-1 before observing y_t. After y_t arrives, calculate:

$v_t = y_t - \mu - a_{t|t-1}$

and evaluate the likelihood contribution using the prior predictive mean and variance. Only after that should y_t be used to update the state.

Rewrite the explanation so the order is:

1. form the prior prediction; 2. observe y_t; 3. compute the innovation and likelihood using the prior; 4. update the state; 5. propagate the state forward.

Check whether the existing code already follows this order before changing any algorithm.

Result:

The explanation was corrected. The existing implementation already used the correct predictive-innovation-update order, so no numerical algorithm was changed.

Correction 2 — Interpretation of Hessian Standard Errors

Original issue:

An earlier explanation referred to the reported uncertainty as "finite-sample Gaussian standard errors." This overstated what the Hessian calculation provides.

Correction prompt:

Recheck the interpretation of the reported parameter standard errors.

They are calculated from the observed-information Hessian of the full Gaussian log likelihood and transformed back to the reported parameter scale using the delta method.

Determine whether these should be described as exact finite-sample standard errors or as model-based asymptotic standard errors.

Revise the written interpretation accordingly, but do not change a correct Hessian implementation merely to change the wording.

Also state clearly that these standard errors are conditional on the model specification and are not automatically robust to model misspecification.

Result:

The wording was changed to "model-based asymptotic Hessian/delta-method standard errors." The existing full-likelihood curvature and parameter-transformation calculations were retained.

Correction 3 — RMSE versus MAE Interpretation

Original issue:

An earlier interpretation described the monthly fit as outperforming the persistence benchmark too broadly.

The actual forecast metrics were:

Monthly fit: RMSE = 2.38659; MAE = 1.76091

Annual-observation fit: RMSE = 2.20921; MAE = 1.60944

Benchmark: RMSE = 2.43691; MAE = 1.67285

Thus, the monthly fit has a lower RMSE than the benchmark but a higher MAE.

Correction prompt:

Recheck the claim that the monthly model outperforms the benchmark using both reported loss functions.

Compare RMSE and MAE separately on exactly the same 913 targets. Do not describe one method as uniformly better if the rankings differ.

Explain briefly why squared-error and absolute-error rankings can differ: RMSE places more weight on large forecast errors, while MAE weights absolute errors linearly.

Also preserve the distinction between a retrospective full-sample comparison and a genuine out-of-sample forecasting exercise.

Result:

The final interpretation states that the monthly fit has lower RMSE but higher MAE than the benchmark. The annual-observation fit has the lowest RMSE and MAE in this sample. The report does not interpret these point estimates as proof of statistically significant or genuine out-of-sample superiority.

Correction 4 — Runnable-Code Dependency Portability

Original issue:

An earlier version of the Python environment specification contained NumPy and SciPy dependencies pointing to temporary local wheel paths from the development machine. That could make the submitted code fail on another machine even though the model code itself was correct.

Correction prompt:

Audit the submitted Python environment for machine-specific dependencies.

Remove any NumPy or SciPy requirement that points to a temporary local path on the development computer. Replace machine-local file references with normal version requirements or, where necessary, a clearly documented platform-specific installation branch.

Do not alter the statistical model or estimation code to solve an environment problem.

After changing the dependency setup, rerun the core estimation tasks and confirm that the numerical results remain unchanged within numerical tolerance.

Result:

The machine-local dependency references were removed from the submission environment. The statistical code was not changed for this issue, and the core numerical results were rechecked after the dependency correction.

Correction 5 — Additional Monte Carlo Check of Parameter Standard Errors

Reason for the additional check:

The required simulation exercise verifies that the estimator can recover known parameters as sample size increases. During review, we added a separate repeated-sampling check to examine whether the reported Hessian standard errors are reasonably calibrated at a sample length close to the historical monthly sample.

This was an additional verification exercise, not a new instructor-required coding task.

> **Additional verification prompt:**
>
> Using the existing monthly state-space model and the same true parameters, run a predeclared Monte Carlo experiment with:
>
> T = 936; B = 200 replications.
>
> Use independent random streams fixed before looking at the results.
>
> For every replication: simulate the monthly model from the stationary prior; estimate the model using the same MLE procedure; calculate the same Hessian/delta-method standard errors; retain the parameter estimates, standard errors, convergence diagnostics, and any failure indicators.
>
> Do not replace failed runs or change seeds after inspecting the output.
>
> Across replications, report for each parameter: the true value, the mean estimate, bias, empirical standard deviation of the estimates, mean reported standard error, mean SE divided by empirical SD, nominal 95% interval coverage.
>
> Use this experiment only as a calibration check of the reported uncertainty. Do not describe it as an additional formal homework requirement.

Result:

All 200 predeclared replications produced parameter-valid and standard-error-valid results with no exceptions.

The mean SE / empirical SD ratios were approximately:

μ: 0.951; φ: 0.914; $\sqrt{Q}$: 1.066; $\sqrt{H}$: 1.006

Nominal 95% interval coverage was approximately:

μ: 0.915; φ: 0.940; $\sqrt{Q}$: 0.940; $\sqrt{H}$: 0.970

These results suggest that the asymptotic Hessian standard errors are reasonably informative but not perfectly calibrated at T = 936, especially for μ and φ. This additional check therefore qualifies the interpretation of the standard errors without invalidating the likelihood implementation.

4. Summary

AI was used as a structured research assistant across the full Homework 1 workflow:

1. specify the monthly state-space model; 2. simulate from the known model; 3. estimate the model on simulated data; 4. estimate the model on historical monthly inflation; 5. construct and estimate the augmented annual-observation model; 6. compare forecasts on the same future-year target; 7. simulate from the fitted model and diagnose economic persistence.

The main material corrections concerned: the timing description of the Kalman innovation; the interpretation of Hessian-based parameter standard errors; the distinction between RMSE and MAE rankings; the portability of the submitted Python environment; an additional repeated-sampling check of standard-error calibration.

The corrections changed explanations, reproducibility, and validation where necessary, while preserving the original data, required sample definitions, model structure, and correctly implemented numerical procedures.